

传感器新一代安全通信协议落地方案

Goal: 将《传感器新一代安全通信协议卷》落地为 CPU2、传感器端、测试工具和 SIL 证据可同步推进的实施方案。

Architecture: 采用分层协议栈：字节流传输层只负责点对点 UART6/RS485 收发，帧层负责 CRC32C、长度、字节序和字段偏移，会话层负责 HELLO 挑战、`session_id`、u32 `seq`、规范化 `safety_param_crc` 和 u16 `config_epoch`，安全监控层负责本地有效年龄、状态诊断、序号连续性和安全快照撤销。周期上报使用固定 44 字节 `FAST_REPORT_FRAME`，控制命令使用 `CTRL_FRAME`；SRS 未明确要求周期上报作为安全输入时，默认采用主站请求响应。

Tech Stack: C11、STM32 HAL UART/DMA、RS485 9600 bps、Python 3 验证脚本、Markdown 认证证据。

1. 落地边界

本方案是新协议落地的长期执行依据。实现按任务逐步修改源码，任何一步完成后都要能单独构建、测试和回退；旧 DSM/LTD 协议路径在新协议通过联调和安全门禁前持续保留。

本方案不构成 SIL 符合性声明。P0 门禁未关闭前，只允许开展原型和协议工具工作，不允许在设计评审、测试报告或对外资料中表述为“已满足 SIL”：

- 整机风险分析已经确定目标安全功能和目标 SIL。
- SRS 已定义最大数据年龄、FTTI、安全状态、锁存/恢复条件和通信危险失效率预算。
- 已确认适用产品标准，并由认证机构接受自研 black channel 安全通信层的评估路线。
- 已明确链路是一个 CPU2 对一个传感器的点对点 RS485；多点总线另行设计仲裁和时隙。

正式协议源文件：

- `docs/01_协议与寄存器/传感器通信协议/传感器新一代安全通信协议卷.md`

本方案覆盖范围：

- CPU2 新协议栈实现。
- 传感器端最小实现要求。
- 9600 bps、200 米 RS485 联调和实时性验证。
- golden frame、故障注入和长稳测试。
- SIL 认证资料需要同步保留的证据。

本方案不覆盖：

- CPU3、Modbus、SI7000 或外部协议的寄存器兼容改造。只有当新传感器状态需要暴露给 CPU3 或外部系统时，才进入对应协议目录和 `DEVICE_PROTOCOL_VERSION` 评估。
- 协议内密钥认证。主动攻击、无线开放接入或非受控接入风险由系统安全方案补充。
- PDF 同步。Markdown 是权威正文，PDF 不作为实施阻塞项。

2. 总体实施顺序

实施顺序必须先测试工具、再纯协议栈、再 UART 接入、再业务接入、最后硬件联调。这样可以避免把 CRC、字节序、状态机和业务测量问题混在一起。

阶段	目标	主要输出
P0 安全需求与协议冻结	冻结 SRS 输入、字段、命令、错误码、失效反应和验收口径	认证路线确认、SRS接口表、协议卷复核记录、字段冻结清单
P1 golden frame	先生成跨端一致的标准帧	JSON 向量、Python 校验脚本
P2 CPU2 纯协议栈	不依赖 UART 的编解码和校验	sensor_safe_* 帧层和会话层
P3 CPU2 UART6 传输	接入 9600 bps RS485 半双工收发	UART6 transport、超时和收发窗口
P4 CPU2 业务适配	提供当前测量流程可调用的新服务接口	新协议 client、快报缓存、双向读数
P5 传感器端实现	传感器按最小状态机实现协议	控制帧命令、44 字节快报、退出上报
P6 联调验证	覆盖正常、异常、长稳和 200 米链路	测试记录、波形/日志、故障注入结果
P7 安全论证资料	形成可追溯证据包，不做未经评估的 SIL 声明	需求追踪、CRC残余错误率、诊断覆盖、验证矩阵、残余风险说明

当前执行状态（2026-07-13）：P1 至 P4 的 CPU2 设备侧范围已落地，包括 51 个控制帧、7 个快报帧、参数摘要机器契约，纯协议栈，UART6 传输，FRAM 启动身份，client/service/monitor，MEASURE_READY 状态、饱和诊断计数、能力门禁、SAFE_SENSOR=14 业务适配和 DSM/LTD 回退。每次 HELLO 后增加配置摘要复核，副作用命令的模糊响应禁止跨新会话自动重放，FRAM 平台 SPI 错误可向上阻断，UART 可越过合法长度但不完整的伪前缀。P5 传感器端尚未实现；P6 已完成七组主机测试和四个 GCC -fanalyzer 单元，并完成 CPU2 clean-first 构建；真实传感器、200 米链路、掉电注入和控制窗口台架尚未执行。P0 中整机 SRS 与认证路线门禁仍未关闭，因此当前代码不得作为“已满足 SIL”的发布依据。

本方案的复选框按证据更新：[x] 只表示对应 CPU2、工具或主机测试动作已有源码/命令证据，不代表真实硬件、传感器端或 SIL 门禁通过。混合条目会在正文中明确仍待验证的另一侧边界。

3. CPU2 文件结构建议

安全协议文件统一放在 LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/，与 DSM、LTD、无线配对和通用传感器业务入口隔离。当前 LTD_MAIN_CPU2/CMakeLists.txt 使用 file(GLOB_RECURSE SERVICE_SOURCES ... Services/*.c)，子目录 .c 文件会自动进入 CMake 构建；CMake 与 STM32CubeIDE 均显式加入 Services/Sensor/SafeProtocol 头文件搜索路径。

文件	职责
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_types.h	协议常量、命令枚举、状态位、错误码、字段偏移、固定长度
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_crc32c.h	CRC32C API 声明
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_crc32c.c	CRC32C Castagnoli 软件实现和测试向量入口

文件	职责
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_frame.h	控制帧和快报帧编解码 API
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_frame.c	小端读写、控制帧封包、控制帧解包、快报帧解包
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_session.h	CPU2 会话、事务、上报流状态结构
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_session.c	session_id、seq、safety_param_crc、config_epoch、stream_seq 校验
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_monitor.h	安全快照、有效年龄、状态诊断和故障反应接口
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_monitor.c	使用本地单调时钟撤销过期快照，按 SRS 输出恢复/故障请求
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_transport_uart6.h	UART6/RS485 传输接口
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_transport_uart6.c	半双工发送、DMA 接收、快报帧扫描、超时处理
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_client.h	CPU2 上层可调用的新协议服务 API
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_client.c	HELLO、SET_MEASURE_MODE、READ_MEAS_COMBO、SET_COMM_MODE、GET_STATUS 等命令
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_service.h	当前 sensor.h 业务接口的适配层
LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_service.c	将新协议测量结果转换成当前测量流程需要的 float 或 RAW 值

执行原则：

- 新协议栈不改旧 dsm_sensor_communication.c 和 ltd_sensor_communication.c 的协议逻辑。
- 初期通过编译开关或运行时传感器类型选择新旧协议，便于回退。
- 业务层可以先保留 Read_Density() 等旧接口作为桥接，CPU2 上层重做后再直接调用 SensorSafeClient_*。

- 旧源码可能是 GBK/936，修改 `.c/.h` 前必须确认编码；新增文件建议使用 UTF-8，但不得把旧文件无意转码。
- C/C++ 注释统一使用 `/* ... */`，不得新增 `//` 注释。

4. 工具和测试文件建议

文件	职责
<code>tools/sensor_safe_protocol_vectors.py</code>	生成 golden frame 和 CRC32C 测试向量
<code>tools/check_sensor_safe_protocol_contract.py</code>	校验协议卷、golden frame、字段长度和 CRC 计算范围一致
<code>tools/check_sensor_safe_cpu2_integration.py</code>	校验 18 个冻结命令均有 CPU2 client 入口，以及 <code>SAFE_SENSOR=14</code> 、安全协议到 LTD/DSM 的探测回退顺序、生产业务分发和串口诊断入口未遗漏
<code>tools/check_sensor_safe_host_tests.py</code>	在临时目录统一编译运行七组 CPU2 主机测试，并对 client/service/transport/identity platform 执行 <code>GCC -fanalyzer</code>
<code>docs/01_协议与寄存器/传感器通信协议/golden_frames/sensor_safe_param_digest_v1.json</code>	<code>param_crc</code> 、 <code>safety_param_crc</code> 规范化字节流和边界向量
<code>docs/01_协议与寄存器/传感器通信协议/golden_frames/sensor_safe_protocol_v1.json</code>	固定 golden frame 数据
<code>docs/01_协议与寄存器/传感器通信协议/传感器新一代安全通信协议验证记录.md</code>	记录 PC 仿真、硬件联调、长稳和故障注入结果
<code>docs/01_协议与寄存器/传感器通信协议/传感器安全通信残余危险失效率论证.md</code>	记录误码模型、CRC能力、帧率、预算分配和残余风险

建议先做 Python golden frame，再做 CPU2 C 代码。Python 工具必须输出以下帧：

- HELLO 请求和响应，以及 nonce、双方启动计数和传感器会话计数不匹配的旧响应重放。
- GET_CONFIG_DIGEST 响应。
- SET_MEASURE_MODE(DENSITY) 请求和响应。
- READ_MEAS_COMBO 请求和响应。
- SET_COMM_MODE(PERIODIC_UPLINK) 请求和响应。
- REPORT_MEAS_FAST 连续 3 帧。
- SET_COMM_MODE(REQUEST_RESPONSE) 请求和响应。
- CRC 错、长度错、非零保留字段、地址错、会话错、`config_epoch` 错、`stream_seq` 重复和跳变帧。
- `param_crc`、`safety_param_crc` 的规范化字节流、字段换序和结构体填充反例。

工具验证命令：

```
py tools\sensor_safe_protocol_vectors.py --write docs\01_协议与寄存器\传感器通信协议\golden_frames\sensor_safe_protocol_v1.json --write-param-digest docs\01_协议与寄存器\传感器通信协议\golden_frames\sensor_safe_param_digest_v1.json
py tools\check_sensor_safe_protocol_contract.py
py tools\check_sensor_safe_cpu2_integration.py
```

```
py tools\check_sensor_safe_host_tests.py
py tools\check_docs.py
```

通过标准：

- golden frame 中每帧 CRC32C 可重复计算一致。
- `FAST_REPORT_FRAME` 固定 44 字节，u32 `stream_seq`、u32 `sample_counter` 和 u16 `config_epoch` 偏移正确。
- 控制帧 `frame_len <= 128`。
- 所有多字节字段按小端序编码。
- 协议卷字段偏移和 JSON 向量字段偏移一致。

5. CPU2 详细任务

Task 1: 协议常量和类型冻结

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_types.h`
- Read: `docs/01_协议与寄存器/传感器通信协议/传感器新一代安全通信协议卷.md`
- ☒ Step 1: 定义协议版本、点对点节点默认 ID、控制帧最大长度、快报固定长度、SOF、`frame_type=0xA1`。
- ☒ Step 2: 定义 `cmd`、`msg_type`、测量模式、通信模式、状态位、诊断位、错误码枚举。
- ☒ Step 3: 定义 `CTRL_FRAME` u32 `seq` 和 `FAST_REPORT_FRAME` 的 u32 `stream_seq`、u32 `sample_counter`、u16 `config_epoch` 偏移宏，偏移必须与协议卷第 5 章一致。
- ☒ Step 4: 定义 CPU2 侧测量快照结构，字段使用整数倍率保存，再由适配层转换给旧业务。
- ☒ Step 5: 编译 CPU2，确保头文件无循环包含。

验收命令：

```
cmake --build build\LTD_MAIN_CPU2
```

验收标准：

- 编译通过。
- `sensor_safe_types.h` 不包含 HAL 头文件。
- 快报偏移宏最后满足 `SENSOR_SAFE_FAST_FRAME_LEN == 44U`，且最后一个 `config_epoch` 字节位于偏移 39。

Task 2: CRC32C 实现

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_crc32c.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_crc32c.c`

- Create or update: `tools/check_sensor_safe_protocol_contract.py`
- ☒ Step 1: 先在 Python 工具中固定 CRC32C 测试向量。
- ☒ Step 2: C 端实现 CRC32C Castagnoli 计算接口。
- ☒ Step 3: 在工具中导出同一输入的期望 CRC。
- ☒ Step 4: 用 CPU2 构建确认没有引入库依赖。
- ☐ Step 5: 后续硬件联调时打印或断点比对 C 端 CRC 与 Python 向量。
- ☐ Step 6: 对 44 字节和最大 128 字节帧分析 CRC32C 的已检测错误能力，记录误码模型和残余危险失效率预算；不得使用 2^{-32} 作为结论。

验收标准：

- 空负载、1 字节、44 字节快报、128 字节控制帧边界均有向量。
- CRC 初值、输入反射、输出异或和字节序在工具和 C 端完全一致。
- 文档中明确 CRC 是错误检测码，不作为认证码；形成可复核的残余错误率计算输入。

Task 3: 控制帧编解码

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_frame.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_frame.c`
- ☒ Step 1: 实现小端 `read_u16/read_u32/write_u16/write_u32` 内部函数。
- ☒ Step 2: 实现 `CTRL_FRAME` 封包，输入为头字段和 payload。
- ☒ Step 3: 实现 `CTRL_FRAME` 解包，校验 SOF、长度、`header_len`、版本、CRC、地址和所有保留位为 0。
- ☒ Step 4: 对 `HELLO` 允许 `session_id=0` 和 `safety_param_crc=0`，其它命令必须由会话层检查。
- ☒ Step 5: 禁止结构体强转，所有字段按偏移逐字节读写。
- ☒ Step 6: CRC、长度或 SOF 错帧只静默丢弃并记本地计数，不依据不可信帧发送 `ERR`。

验收标准：

- 可生成与 Python golden frame 完全一致的 `HELLO`、`READ_MEAS_COMBO`、`SET_COMM_MODE` 请求帧。
- 短帧、超长、CRC 错、地址错、版本错均被拒绝。
- 解包函数不访问 payload 边界外数据。

Task 4: 快报帧解码和数据快照

Files:

- Modify: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_frame.h`
- Modify: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_frame.c`
- ☒ Step 1: 实现 `FAST_REPORT_FRAME` 解码，固定检查 `0xA5 0x5A`、`frame_type=0xA1`、`frame_len=44`。
- ☒ Step 2: 校验快报 CRC32C 覆盖 `frame_type` 到 u16 `config_epoch` 末尾。

- ☒ Step 3: 按偏移读取 `src_id`、`dst_id`、`session_id`、`stream_id`、`stream_seq`、`sample_counter`、`data_age_ms`、状态位、诊断位和测量值。
- ☒ Step 4: 输出整数倍率快照，不在帧层转 `float`。
- ☒ Step 5: 对 u32 `stream_seq`、u32 `sample_counter` 和 u16 `config_epoch` 只输出原始值，连续性由会话层判断。

验收标准：

- 正常 3 帧 golden frame 全部解码成功。
- 任意单 bit 错误导致 CRC 失败。
- `src_id` 或 `dst_id` 错误不进入业务层。

Task 5: 会话和上报流状态机

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_session.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_session.c`
- ☒ Step 1: 定义 CPU2 会话状态：`OFFLINE`、`SESSION_READY`、`MEASURE_READY`、`PERIODIC_ACTIVE`、`RECOVERING`、`FAULT`。
- ☒ Step 2: 保存当前 HELLO 挑战、当前及最近一次已接受的 `session_id`、`sensor_id`、`safety_param_crc`、`scale_version`、`config_epoch`、`stream_id`、最近 `seq`、最近 `stream_seq` 和最近 `sample_counter`；协议停用只清当前会话/事务/流运行态，必须跨停用边界保留传感器启动计数、会话计数和最近会话号，新 HELLO 不得复用停用前会话号。
- ☒ Step 3: 实现双向响应校验：地址、会话、u32 `seq`、`cmd`、`safety_param_crc`、`result_code`、状态位、诊断位和本地有效年龄。
- ☒ Step 4a: 实现快报协议内在严格校验：地址、会话、流 ID、`stream_seq == last + 1`、`sample_counter` 严格前进、`config_epoch`、`STREAM_ACTIVE`、流丢失/缓冲溢出、当前模式通道诊断、质量编码范围和本地有效年龄。
- ☐ Step 4b: SRS 冻结传感器采样率、上报周期容差和最大允许样本增量后，实现 `sample_counter` 最大增量合理性门禁及边界测试；当前不得把“严格前进”表述为“增量合理性已完成”。
- ☒ Step 5: 实现配置变化处理：`config_epoch` 不一致时丢弃快报并要求重新 `GET_CONFIG_DIGEST` 或 `HELLO`。
- ☒ Step 6: 任一跳号、回退或重复使安全上报流失效；后续连续帧不得自行恢复，必须重新启动流或 `HELLO`。
- ☒ Step 7: 完整性、身份、连续性和模式通过但状态/诊断/年龄无效的快报推进已验证序号并刷新通信静默基准，但返回数据无效且不得发布安全快照；通信活性与安全数据可用性分别判定。

验收标准：

- 重复快报、跳号快报、旧会话快报、旧流快报均不会更新安全相关测量值。
- 安全协议停用并回退旧协议后再次探测时，回放停用前 `session_id` 仍返回 `REPLAY_DETECTED`；新的单调会话身份可以正常建立。
- u32 `seq`、`stream_seq`、`sample_counter` 和 u16 `config_epoch` 预计溢出前主动停止业务并重新 `HELLO`，同一会话内不允许回绕复用。

Task 6: UART6/RS485 传输层接入

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_transport_uart6.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_transport_uart6.c`
- Create: `tools/test_sensor_safe_transport_uart6.c`
- Create: `tools/sensor_safe_uart6_stubs/usart.h`
- Create: `tools/check_sensor_safe_host_tests.py`
- Read: `LTD_MAIN_CPU2/Services/Sensor/dsm_sensor_communication.c`
- Read: `LTD_MAIN_CPU2/Core/Src/usart.c`
- ☒ Step 1: 复用现有 UART6 DMA 收发经验，但不要调用 DSM 文本协议的 BCC 或换行等待逻辑。
- ☒ Step 2: 实现控制帧发送前清空旧接收状态，接收 DMA 必须先于发送 DMA 启动，发送后按 `frame_len` 或固定头部长度的接收响应；UART6 主机桩直接记录 DMA 事件顺序并断言接收先于发送。
- ☒ Step 3: 实现快报帧扫描状态机：查找 SOF、预判控制帧 32..128 字节或固定快报 44 字节、校验后交给会话层；产品传输源码的主机测试覆盖噪声、非法长度候选、合法长度但不完整的阻塞前缀、短帧超时和满缓冲。只有后方候选 CRC 正确且完整时才越过阻塞前缀，并饱和记录重同步计数。
- ☒ Step 4: 周期上报期间传感器除自身发送外持续接收，CPU2 只在合法快报 CRC 结束后的确定窗口发送控制帧；9600 bps 下协商窗口不得小于 100 ms。CPU2 窗口门禁已实现，传感器持续接收仍由传感器端和台架验证。
- ☒ Step 5: CPU2 已使用饱和计数分别记录 UART `ORE/FE/NE/PE/DMA/BREAK`、硬件错误总数、超时和短帧，并由 client 分别记录 CRC 错和地址错。HAL/UART6 主机桩已把五类 HAL 错误和 BREAK 同事件注入产品传输源码并断言独立计数；现场诊断日志映射、真实 UART 故障注入和长稳分母记录仍按第 7 章执行，不作为硬件通过证据。
- ☐ Step 6: 固化 DE/RE 切换、接收 DMA 重新装载和本地单调时间戳边界，测得最坏调度抖动并纳入控制窗口。

验收标准：

- 不依赖换行符判断帧边界。
- 9600 bps 下 `REPORT_MEAS_FAST` 收帧不阻塞主测量流程。
- 响应超时按协议卷 9.3 节预算处理。
- 任一 UART 错误不会把旧缓存数据当作新数据。
- 46 字节退出请求在最坏方向切换和调度抖动下仍能在 100 ms 控制窗口内完整接收。

Task 7: CPU2 高层 client

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_client.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_client.c`
- ☒ Step 1: 实现 `SensorSafeClient_Init()`，初始化节点 ID、超时、会话状态和统计计数。

- ☒ Step 2: 实现 `SensorSafeClient_Hello()`：每个新事务生成新的 `cpu_nonce`，校验响应回显的 CPU2 启动计数、nonce、传感器启动计数、会话计数和新 `session_id` 非复用，再保存会话身份与配置摘要。
- ☒ Step 3: 实现 `SensorSafeClient_SetMeasureMode()`。
- ☒ Step 4: 实现 `SensorSafeClient_ReadMeasurementCombo()`。
- ☒ Step 5: 实现 `SensorSafeClient_StartPeriodic()` 和 `SensorSafeClient_StopPeriodic()`；进入 ACK 必须声明 `STREAM_ACTIVE` 并保存采用的流参数，退出 ACK 必须确认请求响应模式、清除流状态、保留配置纪元且所有保留字段为 0，不得从第一帧快报反向学习期望纪元。
- ☒ Step 6: 实现 `SensorSafeClient_PollPeriodic()`，只在会话层校验通过后更新最新快照；首帧截止点按进入响应接收时间加 `start_after_ms + accepted_max_silent_ms` 计算，后续截止点按最近合法快报接收时间加 `accepted_max_silent_ms` 计算，底层接收等待裁剪到剩余预算，并按实际接收时间戳二次拒绝底层迟到后仍返回成功的快报。
- ☒ Step 7: 实现 `SensorSafeClient_GetStatus()` 和 `SensorSafeClient_GetCounters()`，计数采用饱和加一并保留请求、重试、拒绝和接收分母。
- ☒ Step 8: 可信远端 `BAD_SESSION`、`CONFIG_MISMATCH` 进入恢复态并复用服务层单次 HELLO 边界；其它远端故障保持原错误分类。HELLO 通用前缀语义拒绝不得覆盖已记录的具体协议原因。

验收标准：

- 每次只允许一个未完成请求。
- 有副作用命令使用 `seq` 做幂等保护。
- 旧 HELLO 响应、nonce 不匹配、会话计数重复和新挑战返回旧缓存均不能建立会话。
- 周期流截止前的短轮询超时不使流失效，下一帧合法快报仍可接受；首次或后续静默截止点到达时返回 `DATA_STALE` 并立即使流失效。
- 退出周期上报成功后，同一 `session_id` 下可继续双向读取。

Task 8: 当前业务适配

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_service.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_service.c`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_monitor.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/SafeProtocol/sensor_safe_monitor.c`
- Modify with care: `LTD_MAIN_CPU2/Services/Sensor/sensor.c`
- Modify with care: `LTD_MAIN_CPU2/Services/Sensor/sensor.h`
- Read: `LTD_MAIN_CPU2/Application/Src/measure_density.c`
- ☒ Step 1: 定义新协议启用条件，例如编译开关或传感器类型识别结果。
- ☒ Step 2: 让 `Read_Density()` 在 `SAFE_SENSOR=14` 时通过 `service/client` 组合读，输出当前业务仍需要的 `float frequency/density/temp`。
- ☒ Step 3: 保持旧 DSM/LTD 协议路径可回退。

- ☒ Step 4a: 建立隔离的周期安全快照；发布和读取使用本地单调时间计算饱和 `effective_age_ms`，并对通信模式、状态/诊断保留位、`STREAM_ACTIVE`、流故障、当前模式通道诊断和协议内在质量范围执行二次防御性校验。
- ☐ Step 4b: SRS 冻结各物理量绝对范围、变化率、质量阈值和连续有效快照数量后，将其接入安全快照采用策略；当前实现不得宣称已完成系统级物理合理性门禁。显示快照的最终隔离接口也需随整机消费者设计确认。
- ☒ Step 5: 任一关键校验失败时，在当前调度周期撤销安全快照有效标志，旧值不得继续参与安全控制；当前实现限定 CPU2 主循环单调度域，不宣称跨 ISR 原子队列。
- ☒ Step 6: 新增错误映射，把 CRC 错、序号跳变、静默超时、配置不一致、数据过期、诊断异常映射到现有错误日志体系；整机安全反应仍需 SRS 和 FTTI 闭环。
- ☒ Step 7: 串口通信诊断和整机通信测试识别 `SAFE_SENSOR`，通过安全适配器执行组合密度读取；旧 DSM/LTD 诊断分支保持原调用路径。
- ☒ Step 8: 服务层诊断计数采用饱和加一；周期流活动时普通请求/响应业务立即返回会话错误，不在控制窗口外隐式发送新 HELLO。
- ☒ Step 9: 每次初始或恢复 HELLO 后强制读取配置摘要并逐字段核对；只读事务允许一次跨会话恢复，副作用命令在传输/协议结果不确定时禁止自动重放并记录阻断计数。
- ☒ Step 10: FRAM 身份平台层直接检查 SPI 命令、地址和数据阶段的 HAL 状态，完整读写 0x40 字节槽并验证保留区 0xFF；主机平台桩覆盖读写成功、各阶段失败、地址越界和 RNG 状态。

验收标准：

- 不改变未启用新协议时的旧传感器行为。
- 新协议读数输出和协议整数倍率一致。
- 数据无效时不能沿用上一帧安全相关测量值。
- 从故障检测到安全输出动作完成的端到端实测时间小于对应安全功能 FTTI。

6. 传感器端详细任务

传感器端固件不在当前仓库时，按本节模块名在传感器项目中建立同等职责文件。传感器端优先实现最小集合，不引入动态内存、分片、多事务或多上报流。

建议文件：

- `sensor_safe_types.h`
- `sensor_safe_crc32c.c/h`
- `sensor_safe_frame.c/h`
- `sensor_safe_session.c/h`
- `sensor_safe_measure.c/h`
- `sensor_safe_stream.c/h`
- `sensor_safe_uart.c/h`

传感器端任务：

- ☐ 实现控制帧接收状态机：SOF、长度、最大 128 字节、CRC32C、地址、版本、`header_len`。
- ☐ 实现 HELLO：校验 CPU2 挑战，递增本次启动内 `sensor_session_counter`，绑定双方 ID/启动计数/nonce 生成新 `session_id`，并回显 CPU2 启动计数和 nonce。

- ☐ 实现 `GET_CONFIG_DIGEST`，按规范化字节流返回摘要 schema、当前倍率、标定、参数表版本、配置纪元和配置 CRC；禁止对 C 结构体内存直接求 CRC。
- ☐ 实现 `SET_MEASURE_MODE`，模式切换期间置 `MODE_SETTLING`，稳定后置 `MODE_MATCH`。
- ☐ 实现 `READ_MEAS_COMBO`，从同一次采样快照返回温度、密度、频率和质量值。
- ☐ 实现 `SET_COMM_MODE(PERIODIC_UPLINK)`，校验 `report_cmd=REPORT_MEAS_FAST`、`reserved0=0`、周期、静默时间和 `stream_id`。
- ☐ 实现固定 44 字节 `FAST_REPORT_FRAME`，每帧更新 u32 `stream_seq`、u32 `sample_counter`、`data_age_ms`、状态位、诊断位和 u16 `config_epoch`。
- ☐ 实现 `SET_COMM_MODE(REQUEST_RESPONSE)`，停止当前上报流并返回 ACK。
- ☐ 实现 `GET_STATUS`、PING、RESET_SESSION。
- ☐ 实现错误分类：`BAD_CRC`、不可信长度只进入本地诊断计数；`BAD_ARGUMENT`、`BAD_STATE`、`UNSUPPORTED_CMD`、`CONFIG_MISMATCH` 在帧身份可信时可返回错误响应。
- ☐ CRC、SOF 或长度错误只静默丢弃和计数；仅对完整性、地址、会话和事务可信的业务错误发送 `ERR`。
- ☐ 参数写入使用暂存区和原子双副本提交，覆盖提交中断电、回读失败、旧副本回退和提交后旧流失效。

传感器端验收标准：

- 周期上报期间仍能接收退出命令。
- 配置变化时停止当前上报流或使 `config_epoch` 变化，CPU2 不会继续使用旧流数据。
- 快报帧只按固定 44 字节发送，不根据保留字段变长。
- 同一次快报中的温度、密度、频率来自同一测量快照或带有明确 `sample_counter`。

7. 联调和验证计划

7.1 PC 仿真先行

- ☒ 主机 fake transport 模拟传感器，CPU2 协议库发送 `HELLO`、`READ_MEAS_COMBO`、`SET_COMM_MODE` 及其余冻结命令。
- ☒ 模拟器按 golden frame 返回正常响应并逐字节核对请求。
- ☒ CPU2 主机测试已注入 CRC 错、短长度、错误 SOF/版本/头长/消息类型、非零保留字段、重复/跳号、旧会话、错地址、`config_epoch` 错、质量越界、缺少 `STREAM_ACTIVE`、流丢失/缓冲溢出和当前通道诊断；UART6 产品源码另覆盖噪声/非法长度后的 SOF 重同步、短帧、满缓冲、RX/TX 启动失败、中止和 HAL 错误分类。任意 bit 穷举及真实 UART/DE/RE 时序注入仍留给板级测试。
- ☒ CPU2 主机测试已重放或构造旧 `HELLO` 响应，覆盖 nonce、CPU2 启动计数、传感器启动计数、传感器会话计数和新 `session_id` 复用；传感器端模拟器与实物对拍仍未执行。
- ☒ 主机测试注入接收处理延迟，验证本地 `effective_age_ms` 而不是只相信传感器 `data_age_ms`。
- ☒ CPU2 侧记录 transport、protocol、wire 错误和饱和诊断计数，保留每种异常的拒绝原因与统计分母。

通过标准：

- 所有异常帧不更新安全相关测量值。
- 所有正常帧能还原成协议卷定义的工程值。
- 错误计数和日志可追溯到故障类型。
- CRC/长度错帧不会触发在线错误响应，避免噪声下响应风暴或半双工碰撞。

7.2 板级短线联调

- ☐ CPU2 和传感器短线 9600 bps 通信。
- ☐ 完成上电 `HELLO`、设置密度模式、双向读取、进入周期上报、退出周期上报、继续双向读取。
- ☐ 抓取串口原始 HEX，与 golden frame 对比字段位置和 CRC。

通过标准：

- `READ_MEAS_COMBO` 请求和响应在普通请求超时预算内完成。
- `REPORT_MEAS_FAST` 周期稳定，1000 ms 周期下无长期漂移。
- 退出上报后不再接受旧 `stream_id` 数据。

7.3 200 米 RS485 长线验证

- ☐ 使用目标线缆、终端电阻、偏置电阻和实际接地方式搭建 200 米链路。
- ☐ 9600 bps 连续运行至少 24 小时，周期 1000 ms；该结果只作为稳定性证据，不替代残余危险失效率论证。
- ☐ 记录总字节数、总帧数、CRC 错、短帧、UART `ORE/FE/NE/PE/BREAK`、静默超时、重试次数、恢复次数和退出上报成功率，所有比率都保留分母。
- ☐ 在不变更项目基线的前提下，可追加 19200 bps 和 38400 bps 探索性测试，结果只作为现场能力参考。

通过标准：

- 9600 bps 为正式通过项。
- u32 序号接近溢出时能在复用前停止业务并重新 HELLO，不测试或允许同一会话内回绕续用。
- 任一通信异常都触发数据无效或恢复流程，不产生未标识的旧数据。

7.4 故障注入矩阵

类别	注入方式	期望结果
数据损坏	翻转任意 bit	CRC 错，丢弃
截断	少发 1 到 10 字节	长度或超时失败，丢弃
粘包	两帧连续无间隔	能按 SOF 和长度重新同步
错地址	改 <code>src_id</code> 或 <code>dst_id</code>	丢弃，不更新数据
旧会话	使用旧 <code>session_id</code>	丢弃
重放	重复旧 <code>stream_seq</code>	丢弃，安全上报流失效
丢帧	跳过一个 <code>stream_seq</code>	当前帧不进入安全输入，安全上报流失效并重新启动
冻结	<code>sample_counter</code> 不前进	当前已实现数据无效；最大增量异常待 SRS 冻结采样率约束后实现
配置变化	改 <code>config_epoch</code>	丢弃并重新确认配置
数据过期	<code>data_age_ms</code> 超限	不作为安全相关输入
本地延迟	接收后延迟消费或阻塞任务	<code>effective_age_ms</code> 超限，旧快照原子失效
HELLO 重放	回放旧响应或篡改 challenge 回显	不建立会话
停用后 HELLO 重放	建立会话后停用安全协议，再回放停用前 <code>session_id</code>	回放历史仍有效，不建立旧会话；新会话号可建立
首次快报静默	进入 ACK 后在 <code>start_after_ms + accepted_max_silent_ms</code> 截止点前不发快报，并给 Poll	实际等待裁剪到剩余预算；截止点返回 <code>DATA_STALE</code> ，迟到成功帧也被拒

类别	注入方式	期望结果
	传入更长超时；另注入底层越过截止点后才返回成功帧	绝，流失效
周期短轮询与后续静默	合法快报后先执行小于剩余预算的短轮询超时，再在下次轮询提供合法帧；随后超过最近合法帧的最大静默	短轮询只返回传输超时且流保持；下一帧被接受；最终截止点返回 <code>DATA_STALE</code> 并使流失效
控制窗口	临界时刻发送 46 字节退出请求并叠加最大调度抖动	100 ms 窗口内完整接收；否则按静默和恢复策略失效
参数提交掉电	在写入、回读和有效标记切换阶段分别复位	只加载最后一个完整有效副本，旧流失效
传感器故障	<code>DATA_VALID=0</code> 或诊断位置位	不更新安全相关测量值

8. SIL 证据同步要求

实现过程中必须同步积累下列证据，避免代码完成后再补认证资料。

证据	产物	对应标准主题
需求追踪	协议卷第 1 章和实现任务追踪表	IEC 61508-1 安全生命周期
架构分层	CPU2 文件结构和传感器端文件结构评审记录	IEC 61508-3 软件架构
接口明确性	字段偏移、类型、字节序、倍率表、golden frame	IEC 61508-3 接口规格
black channel 防护	CRC、地址、会话、序号、数据年龄、状态诊断验证记录	IEC 61784-3
通信失效处理	故障注入矩阵执行记录	IEC 62280
配置管理	<code>scale_version</code> 、 <code>safety_param_crc</code> 、 <code>config_epoch</code> 变化测试	IEC 61508-1 配置管理
残余风险	CRC32C 充分性、主动攻击边界、目标 SIL 来源说明	IEC 61508、IEC 62443
安全反应	快照撤销到执行器安全动作的端到端 FTTI 记录	IEC 61508-2 故障反应和安全状态
物理链路	点对点拓扑、终端/偏置/隔离、DE/RE、UART硬件错误和 EMC记录	黑通道假设与硬件集成证据

认证前必须完成：

- CRC32C 对最大帧长、误码率和目标危险失效率的充分性论证，禁止直接用 `2^32` 代替帧长和错误模型分析。
- 整机 SRS 给出最大允许数据年龄、最大静默时间、FTTI、安全状态、锁存和恢复条件。
- 诊断覆盖率估算和故障注入结果闭环。
- 现场参数修改权限、审计、备份和回滚流程。
- 认证机构确认适用标准和自研 black channel 安全通信层评估路线。

9. 版本、提交和流程文档影响

普通实现阶段不主动升 CPU2/CPU3 固件版本。提交或发布前按最终 diff 判断：

- 只新增协议文档和工具，不改变固件输出：通常不升级固件版本。
- CPU2 新增协议栈但未启用：评估 CPU2 固件版本，通常不改变 CPU2/CPU3 共享 `DEVICE_PROTOCOL_VERSION`。
- CPU2 默认启用新传感器协议：必须升级 CPU2 固件版本，并更新 `CHANGELOG.md` 和版本改动与测试方案。
- 新状态、诊断或测量值暴露到 CPU3/Modbus/SI7000：必须评估 `DEVICE_PROTOCOL_VERSION`，同步 CPU2/CPU3 协议文档和变更记录。
- `SAFE_SENSOR=14` 复用现有 `sensorType` 字段，`DeviceParameters` 结构大小和 `DEVICE_PARAM_VERSION=3` 均不变，升级不会仅因本改动恢复出厂参数；旧固件不识别值 14，降级前必须把传感器类型恢复为旧固件支持的类型，且旧固件不能驱动安全协议传感器。
- 若最终 diff 改变传感器通信流程、命令分发、测量结果上报或异常恢复路径，必须同步 `docs/00_程序流程导航/CPU2/09_传感器与无线通信.html` 及对应业务流程图。

提交前建议命令：

```
py tools\check_sensor_safe_protocol_contract.py
py tools\check_docs.py
cmake --build build\LTD_MAIN_CPU2
py tools\check_commit_subject_versions.py --range origin/MAIN..HEAD
```

10. 执行切分建议

建议按以下小批次执行，每批次完成后单独评审。

Batch A: 协议工具和 golden frame

- ☒ 新增 Python golden frame 生成工具。
- ☒ 新增 golden frame JSON。
- ☒ 新增协议契约检查脚本。
- ☒ 运行 `py tools\check_sensor_safe_protocol_contract.py`。
- ☒ 运行 `py tools\check_docs.py`。

完成标准：不需要硬件即可证明协议卷、字段偏移、HELLO challenge、规范化配置 CRC 和 golden frame 一致；此结果不构成 SIL 充分性结论。

当前门禁还会解析 CPU2 `sensor_safe_types.h`，将帧常量、全部字段偏移、命令/模式枚举、标志位、状态/诊断/能力位、线上错误码和允许掩码与 golden JSON 逐项比较；C 侧符号新增、删除或数值漂移均会失败。

Batch B: CPU2 纯协议栈

- ☒ 新增 `sensor_safe_types`。
- ☒ 新增 `sensor_safe_crc32c`。
- ☒ 新增 `sensor_safe_frame`。
- ☒ 新增 `sensor_safe_session`。
- ☒ 新增 `sensor_safe_monitor`。
- ☒ 构建 CPU2。

完成标准：不接 UART，只用 golden frame 验证编解码、挑战响应、严格序号、配置摘要和安全快照撤销。

Batch C: CPU2 UART6 传输和 client

- ☒ 新增 `sensor_safe_transport_uart6`。

- ☒ 新增 `sensor_safe_client`。
- ☒ 对接 UART6 DMA 和超时。
- ☒ 完成主机 fake transport 模拟传感器联调；真实串口模拟器和硬件台架仍归 P6。

完成标准：CPU2 可和 PC 模拟器完成完整握手、双向读数、进入上报、退出上报。

Batch D: CPU2 业务适配

- ☒ 新增 `sensor_safe_service`。
- ☒ 在 `sensor.c` 中增加新协议选择入口。
- ☒ 保留旧协议回退路径。
- ☒ 更新错误日志映射。

完成标准：当前 `SinglePoint_ReadSensor()` 路径可通过新协议拿到密度、温度、频率，异常时不会使用旧数据。

Batch E: 传感器端最小协议

- ☐ 实现控制帧解析和 CRC。
- ☐ 实现核心命令。
- ☐ 实现固定 44 字节快报。
- ☐ 实现退出上报。
- ☐ 通过 golden frame 对拍。

完成标准：传感器端和 Python/CPU2 golden frame 字节级一致。

Batch F: 硬件联调和认证证据

- ☐ 短线联调。
- ☐ 200 米 RS485 长线 9600 bps 联调。
- ☒ 主机帧、会话、服务和快照故障注入已形成自动化记录。
- ☐ 真实 UART、掉电、长线和控制窗口故障注入。
- ☐ 长稳测试。
- ☒ 已形成 CPU2 主机测试、契约检查和 clean-first 构建验证记录。
- ☐ 补齐真实硬件、FTTI、诊断覆盖率和残余危险失效率后形成 SIL 系统级追踪资料。

完成标准：协议满足 9600 bps 实时性和控制窗口预算，异常帧不进入安全相关测量值，端到端安全反应小于 SRS FTTI，认证证据可追溯；最终 SIL 结论由整机评估和认证机构给出。

11. 关键风险和控制措施

风险	影响	控制措施
CRC32C 参数不一致	两端互不兼容	Python golden frame 先行，C 端逐帧对拍
快报偏移错误	CPU2 解析出错 误密度或温度	固定偏移宏，禁止结构体强转
周期上报占满 9600 bps 链路	影响退出命令和 实时性	安全输入默认主站轮询；启用周期上报时采用 1000 ms 周期和不小于 100 ms 的 <code>accepted_control_guard_ms</code>
会话或序号复用	旧响应或旧上报 被误接受	HELLO challenge 回显，u32 序号同一会话禁止回绕，预计溢出前重新 HELLO

风险	影响	控制措施
<code>sample_counter</code> 冻结未发现	使用冻结数据	当前已实现 u32 计数严格前进和本地有效年龄；采样率最大增量门禁待 SRS 提供定量边界
配置变化后仍按旧倍率解释	产生危险测量值	<code>safety_param_crc</code> 和 <code>config_epoch</code> 双重绑定
配置 CRC 两端口径不同	错配置未检出或无法通信	固定规范化字节流、独立 Python golden vector、禁止结构体内存求 CRC
50 ms 控制窗口不足	退出命令截断或碰撞	9600 bps 最小 100 ms，实测方向切换和最大调度抖动
只相信传感器数据年龄	传感器冻结时旧值仍显示新鲜	CPU2 本地单调时钟计算饱和 <code>effective_age_ms</code> ，叠加状态和计数检查
旧协议路径被误破坏	现场旧传感器不可用	新协议独立文件，旧路径保留，启用条件显式
旧源码编码被转码	中文注释或字符串乱码	改旧 <code>.c/.h</code> 前确认编码，避免全文件重写
认证资料滞后	实现完成后无法追溯	每批次同步 golden frame、测试记录和评审结论

12. 最小可交付定义

第一轮可交付必须同时满足：

- P0 已冻结目标安全功能、SRS接口、FTTI、通信危险失效率预算和认证路线；否则交付只能标记为非认证原型。
- CPU2 能通过带 challenge 回显的新协议完成 `HELLO`、`SET_MEASURE_MODE(DENSITY)`、`READ_MEAS_COMBO`。
- CPU2 能进入 `PERIODIC_UPLINK`，接收连续 `REPORT_MEAS_FAST`，并正常退出后继续双向通信。
- 快报固定 44 字节，9600 bps 下 1000 ms 周期稳定。
- CRC 错、长度错、非零保留字段、地址错、HELLO 重放、旧会话、旧流、跳号、`config_epoch` 错、传感器年龄或本地有效年龄超限均不会更新安全相关测量值。
- 安全快照撤销到整机安全动作的实测时间小于 SRS FTTI，恢复前完成重新握手、摘要确认和规定数量的连续有效快照。
- 旧 DSM/LTD 协议路径仍可回退。
- golden frame、故障注入记录、长线测试记录和标准追踪矩阵同步归档。

13. CPU2 设备侧当前落地状态

以下状态表以 2026-07-13 工作区源码为准，优先于前文用于执行拆分的历史复选框：

范围	当前状态	代码或证据
帧和 CRC	已实现	<code>sensor_safe_crc32c</code> 、 <code>sensor_safe_frame</code> 、协议契约检查器
会话与 client	已实现	<code>sensor_safe_session</code> 、 <code>sensor_safe_client</code> ；新事务 u32 seq，事务内重试复用序号并置 <code>RETRY_FRAME</code> ；协议停用保留本次 CPU2 启动内的回放历史

范围	当前状态	代码或证据
启动身份	已实现	FRAM A/B 地址 0x1100/0x1140，单槽 0x40 字节，写后回读和双副本镜像；较新 generation 必须对应严格更大的 boot_counter
UART6 传输	已实现并完成主机行为测试	DMA 接收先启动、同步发送、SOF/长度重同步、取消、超时和诊断计数；HAL/UART6 桩直接执行产品源码，覆盖错误与清理出口
业务服务	已实现	HELLO、模式切换、密度/温度/频率、液位频率、水位电容和姿态单位换算
自动恢复	已实现并按副作用分级	每次 HELLO 后先复核配置摘要；只读事务传输/协议错误后至多一次新 HELLO 和一次业务重试；副作用命令的模糊结果禁止跨会话自动重放，可信 BAD_SESSION/CONFIG_MISMATCH 仍可恢复；尝试、成功和阻断分别饱和计数
周期快照	协议内在门禁已实现，SRS策略待补	传感器采用值校验、通信活性与安全数据可用性分离、首次/后续最大静默期限、接收等待预算裁剪、迟到成功帧时间戳二次拒绝、100 ms 最小控制窗口、双缓冲发布、过期/错序/退出即时撤销；发布 API 二次拒绝模式、状态、诊断、流故障和质量异常；物理范围/变化率待 SRS
旧协议兼容	已实现	识别顺序为 CH9141 链路、安全 HELLO、LTD/V2、DSM；安全协议失败释放 UART6 后回退
持久化类型	已实现，待提交前协议升版	SAFE_SENSOR=14 复用现有 sensorType 字段；DeviceParameters 结构大小和 DEVICE_PARAM_VERSION=3 不变，不因升级清参；CPU2/CPU3 DEVICE_PROTOCOL_VERSION 需要在提交阶段由 15 同步升级到 16；降级到旧固件前必须恢复旧固件支持的传感器类型
主机测试	七组和四个静态分析单元已通过当前实现验证	check_sensor_safe_host_tests.py 固定每组产品源码依赖并统一执行；新增 identity platform 组直接执行产品平台源码，覆盖 SPI 失败传播、完整槽读写和 RNG；service 覆盖 HELLO 后摘要门禁及副作用恢复阻断；transport 覆盖合法长度不完整前缀后的 CRC 完整帧重同步
CPU2 集成契约	已通过	静态检查 18 个冻结命令的 client 入口、SAFE_SENSOR=14、安全→LTD/V2→DSM 探测回退、生产测量分发、串口诊断入口和适配器故障边界
实机和认证	未完成	真实安全传感器、200 米 9600 bps、硬件 RNG、FRAM 掉电、控制窗口、CH9141 切换和旧协议实物回退

版本影响：这属于 CPU2 新功能级变更，提交阶段预计升级 CPU2 MINOR；SAFE_SENSOR=14 又改变 CPU2/CPU3 共享枚举语义，因此提交前还必须同步 CPU3 协议版本、协议变更记录、版本资料和相关流程页面。当前按长期实现阶段约束保持 CPU2 V2.0.0.0、DEVICE_PROTOCOL_VERSION=15，不提前升版。